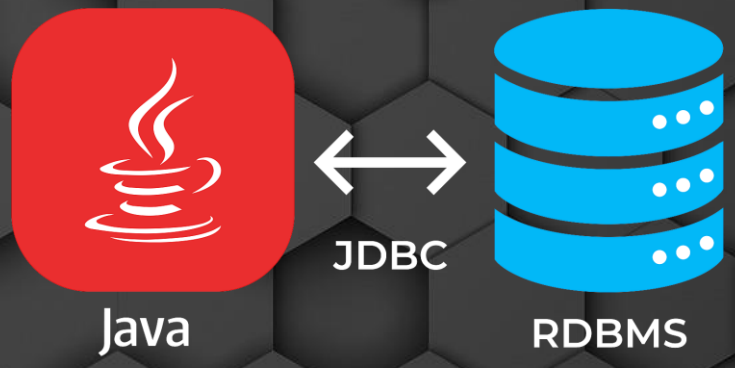
Capa Modelo con JDBC

Desarrollo Entorno Servidor

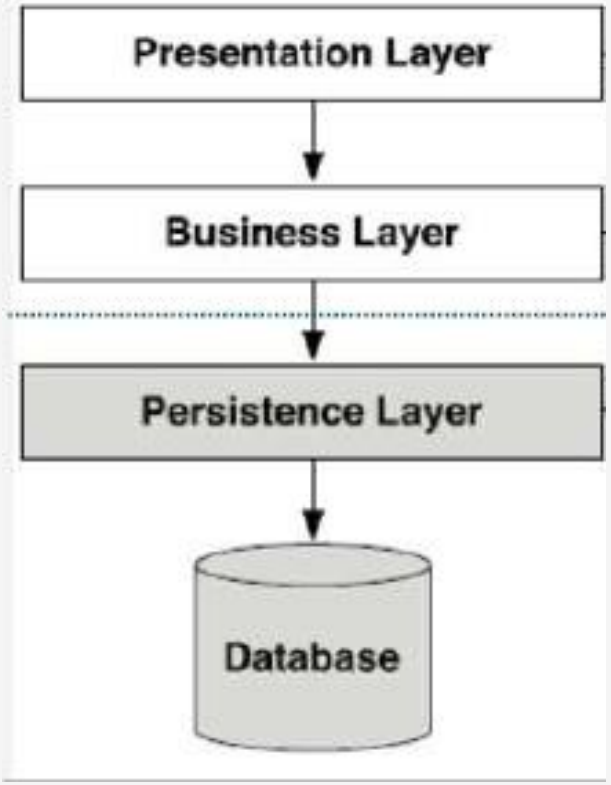


Índice

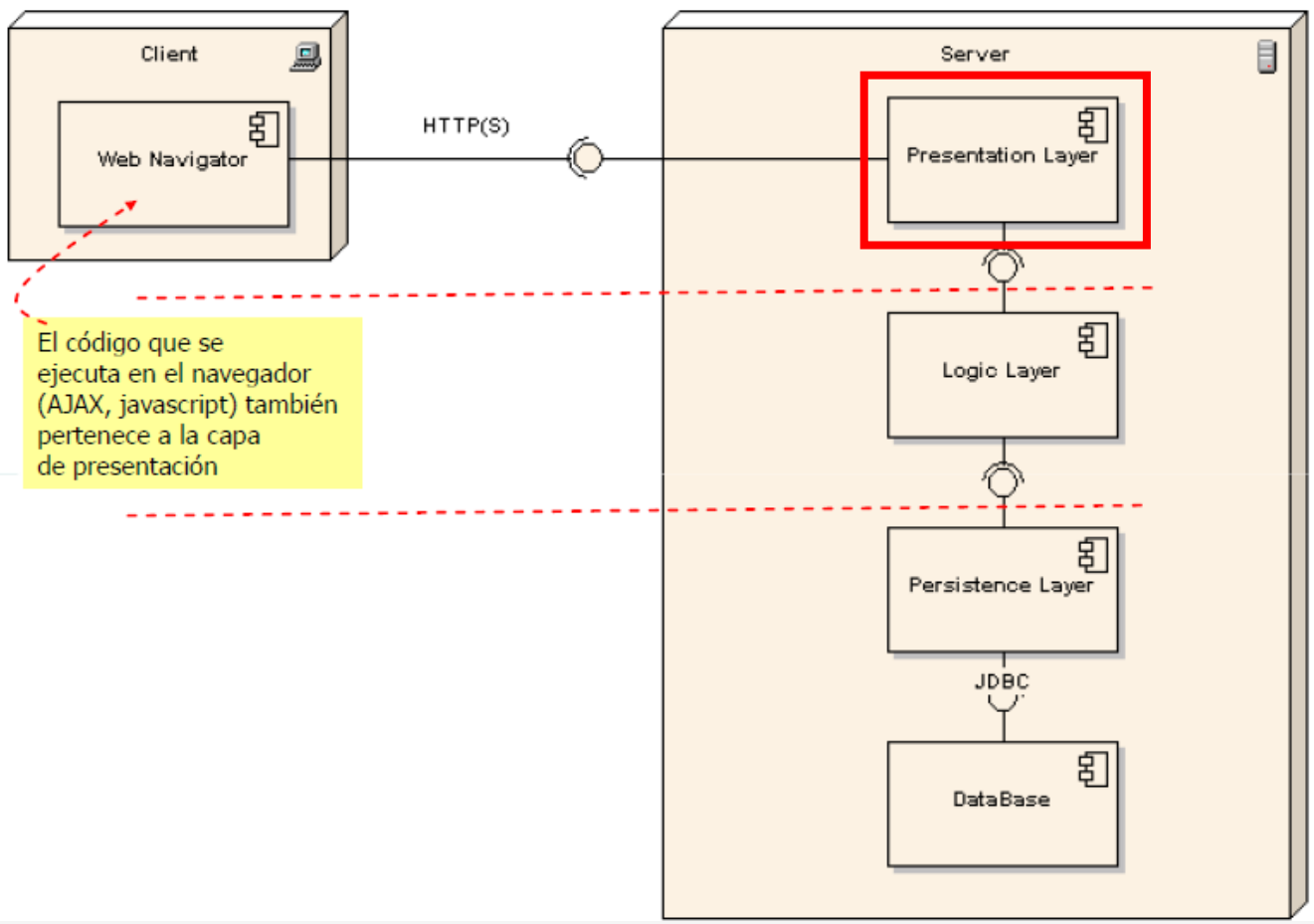
1. Arquitectura en Capas
2. DAO
   1. Patrón factoría
3. JDBC
4. Ejemplo JDBC Puro
5. Spring
6. JDBC Template

1.-Arquitectura en Capas

- La arquitectura define los componentes hardware y software de una aplicación y cómo se relacionan entre ellos.
- Las aplicaciones web tienen una arquitectura de capas.
- Las capas se comunican a través de interfaces con la inferior
- Así conseguimos que sean independientes y reemplazables

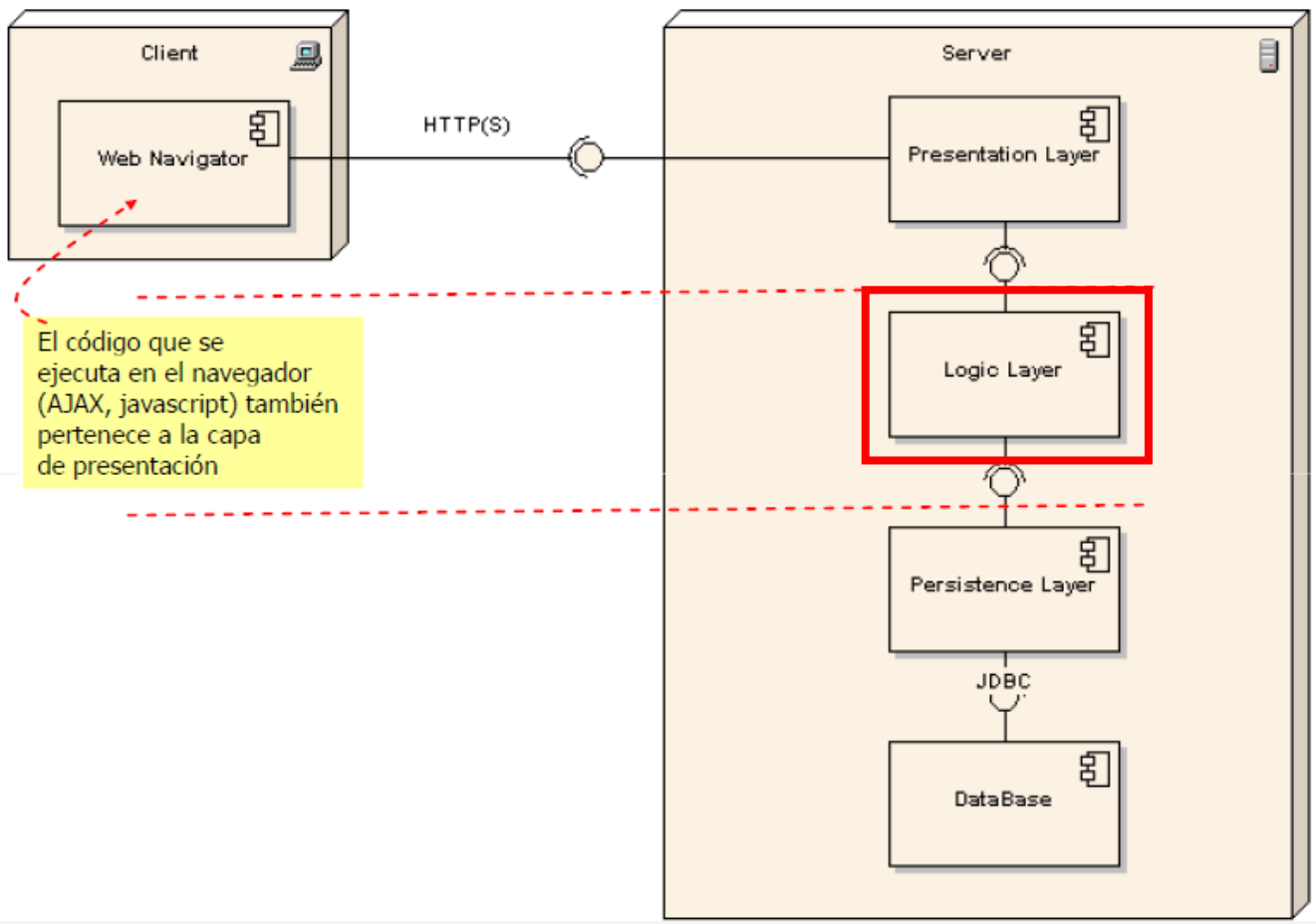


1.-Arquitectura en Capas
Capa de presentación (GUI):



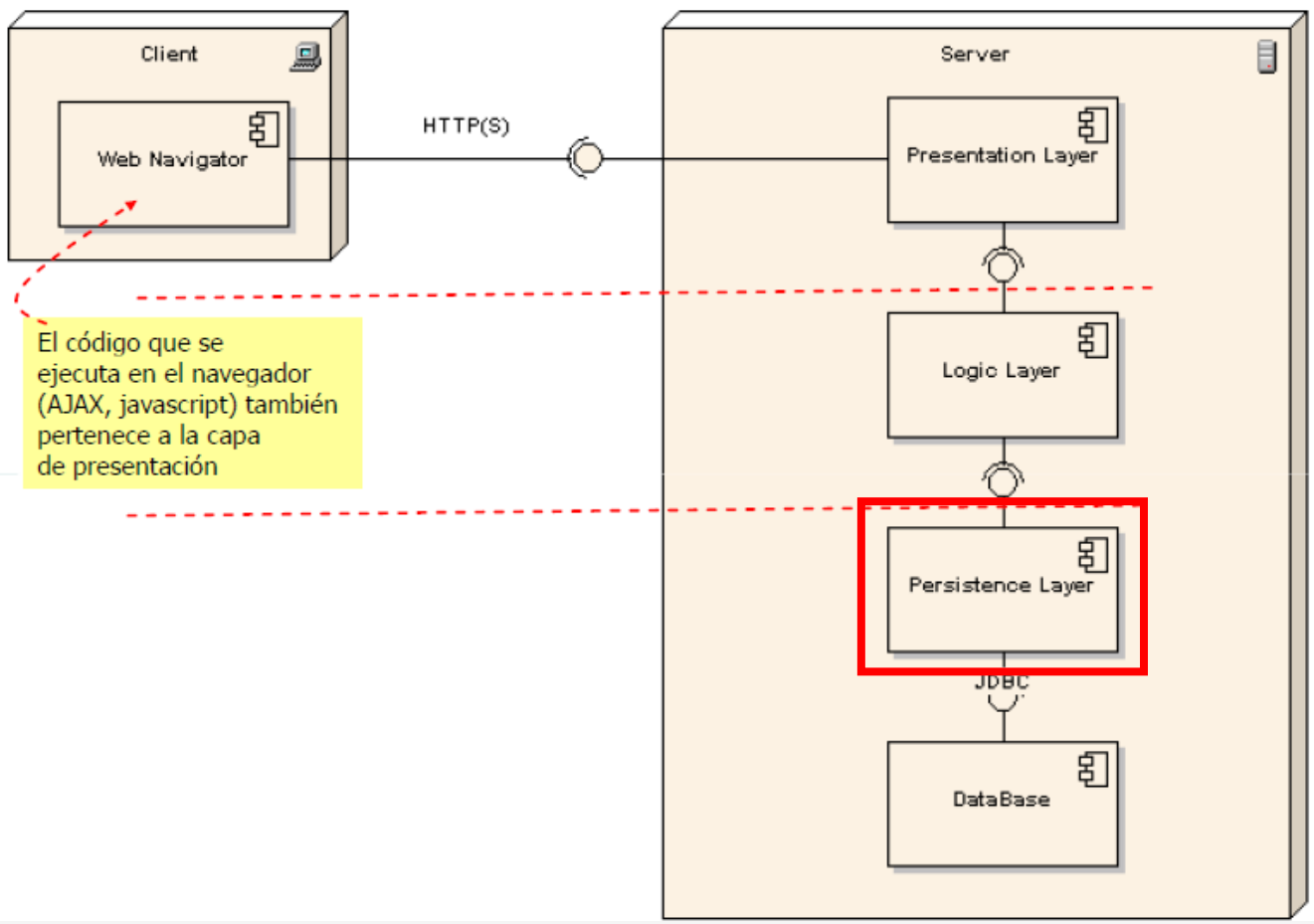
- **Capa de presentación (GUI):**
  - Se encarga de la interacción con el usuario: solicitar, mostrar, formatear, internacionalización de datos
  - Informar adecuadamente de errores de la aplicación
  - En una aplicación web parte esté en el servidor y parte es resuelta por el navegador (AJAX)
  - Es habitual utilizar un patrón **MVC (Model View Controller, ej: framework struts)**

1.-Arquitectura en Capas
Capa de presentación (GUI):



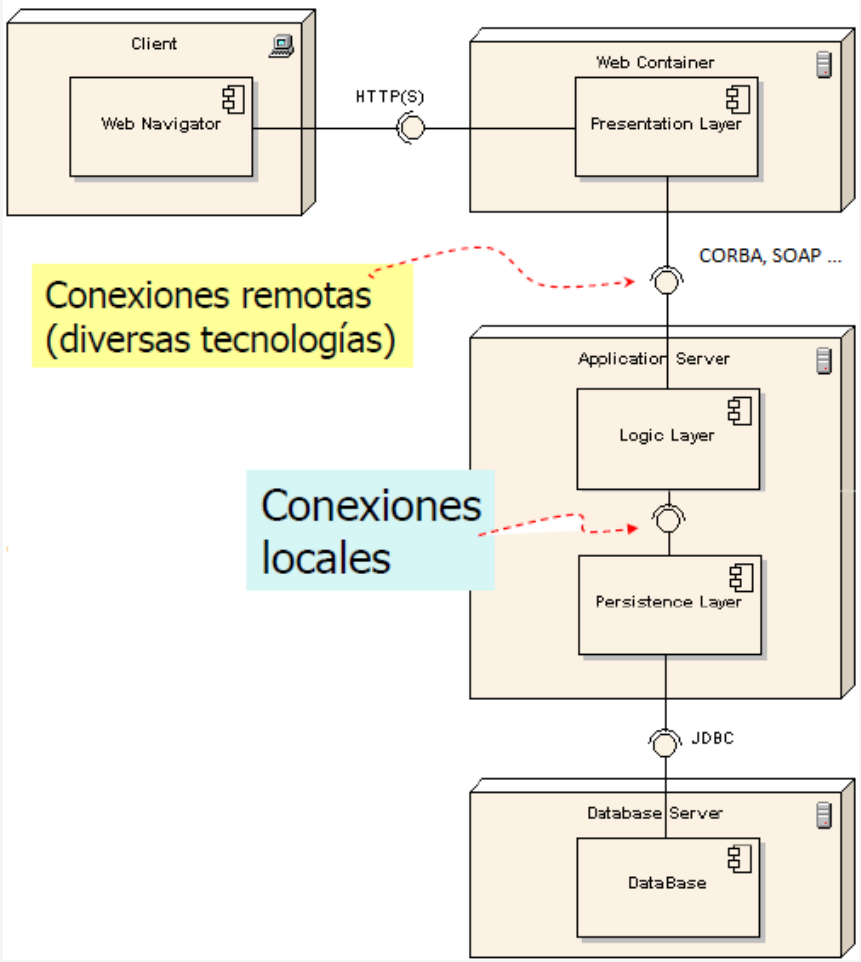
- **Capa de negocio**
  - Resuelve las funcionalidades para las que se ha diseñado la aplicación ofreciéndolas como servicios: Clases con métodos que implementan estas funciones (ej: un servicio web)
  - Para ello necesitamos objetos que representen las entidades de nuestra aplicación
  - Para ello accede a la capa de persistencia y envía/recibe datos de la capa de presentación siendo independiente de ambas

1.-Arquitectura en Capas
Capa de presentación (GUI):



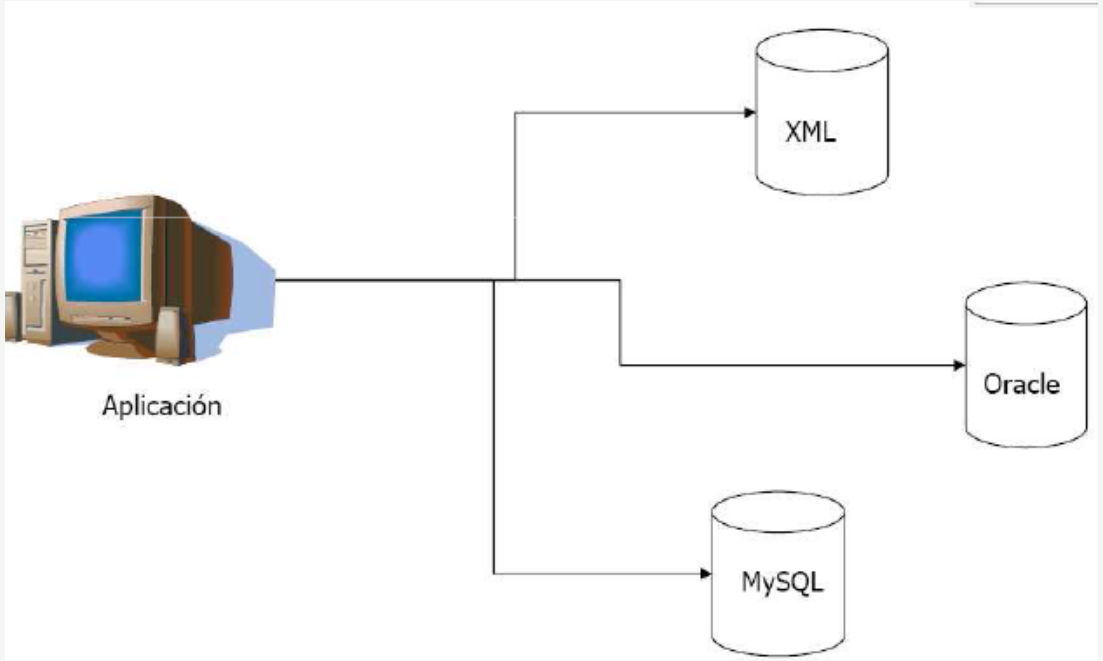
- **Capa de persistencia**
  - La persistencia de datos puede implementarse con múltiples y diferentes tecnologías: ficheros, gestores de BD relacionales diferentes: Oracle, mysql… , BD no relacionales
  - La implementación de la persistencia tiene que ser "transparente" para las capas de negocio y presentación
  - Esta independencia de la implementación concreta se consigue mediante **un patrón DAO**

1.-Arquitectura en Capas



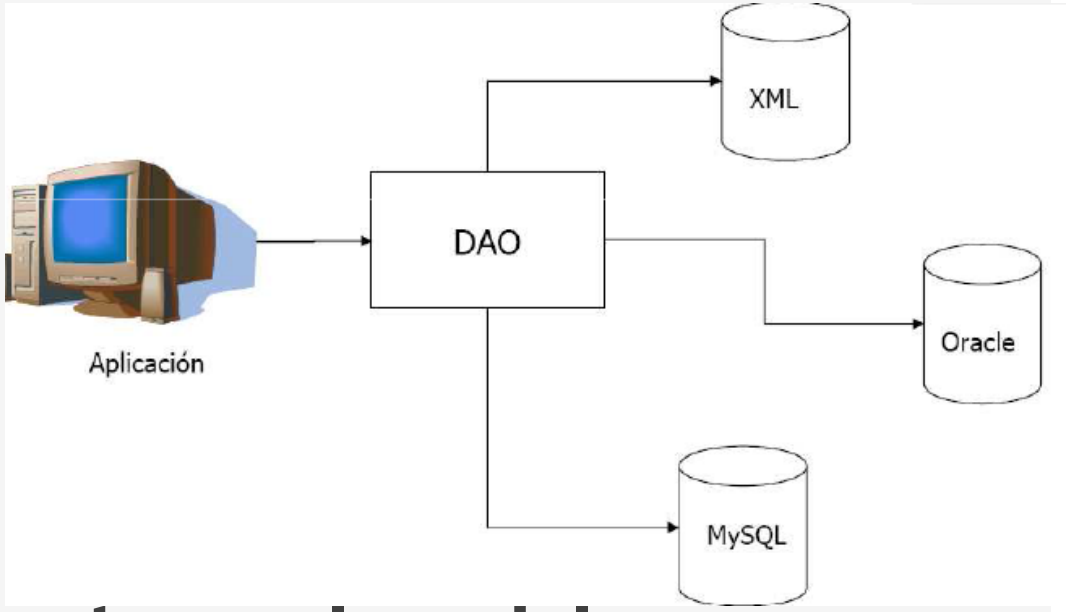
2.-Arquitectura en Capas - Patrón DAO

- Problema:
  - Se necesita independencia del sistema de persistencia
    - BD Relacional, BD orientada a objetos, ficheros, XML, BD XML, serialización …
  - Se debe acceder a varios sistemas desde la misma aplicación
    - Las APIS son diferentes



2.-Arquitectura en Capas - Patrón DAO

- Solución:
  - DAO: Data Access Object proporciona una interfaz única de acceso a los datos independientemente de cómo estén almacenados
  - Proporciona independencia a la lógica del negocio del acceso a datos
  - Habitualmente un objeto DAO por cada entidad del modelo



Para profundizar en un DAO debemos tener claro del funcionamiento del patrón factoría.

2.1.- Patrón Factoría.

Una **factoría** es un objeto que maneja la creación de otros objetos. Las **factorías** se utilizan cuando la creación de un objeto implica algo más que una simple instanciación.
Los siguientes ejemplos son casos donde una factoría puede ayudar:
Es necesario acceder a algún recurso para la creación y configuración de un objeto.

No conocemos hasta el momento preciso de la instanciación qué tipo concreto de objeto se va a instanciar.

En estos casos, en lugar de que sea el propio objeto quien se encargue de todos los aspectos relativos a la creación, se crea otro objeto que lo haga. De esta manera, se libera al objeto que va a ser creado de aquellas responsabilidades que no le corresponden pero que son necesarias para su creación, manteniendo su independencia.

2.1.- Ejemplo Patrón Factoría

Declaramos un interfaz Figura:

```
public interface Figura {
 public double getArea();
}
```

Declaramos dos objetos que instancia de este interfaz.

```
public class Cuadrado implements Figura {
 double lado;

 public Cuadrado(double lado) {
  this.lado = lado;
 }

 public double getArea() {
  return lado * lado;
 }
}
```

```
public class Circulo implements Figura {
 double radio;

 public Circulo(double radio) {
  this.radio = radio;
 }

 public double getArea() {
  return (3.14 * radio * radio);
 }
}
```

2.1.- Ejemplo Patrón Factoría (II)

Declaramos un clase Factoría que en función de los parámetros decide crear un objeto u otro.

```
public class FiguraFactory {
 public final static int CUADRADO = 0;
 public final static int CIRCULO = 1;

 public static Figura getFigura(int tipo, double lado) {
  switch (tipo) {
   case CUADRADO:
    return new Cuadrado(lado);
   case CIRCULO:
    return new Circulo(lado);
  }
  return null;
 }
}
```

Y finalmente…

```
public class Principal {
 public static void main(String[] args) {
  int tipo = Integer.parseInt(args[0]);
  double lado = Double.parseDouble(args[0]);
  Figura figura =
   FiguraFactory.getFigura(tipo, lado);
  System.out.println("El area de la figura es: " + figura.getArea());
 }
}
```

2.- Patrón DAO

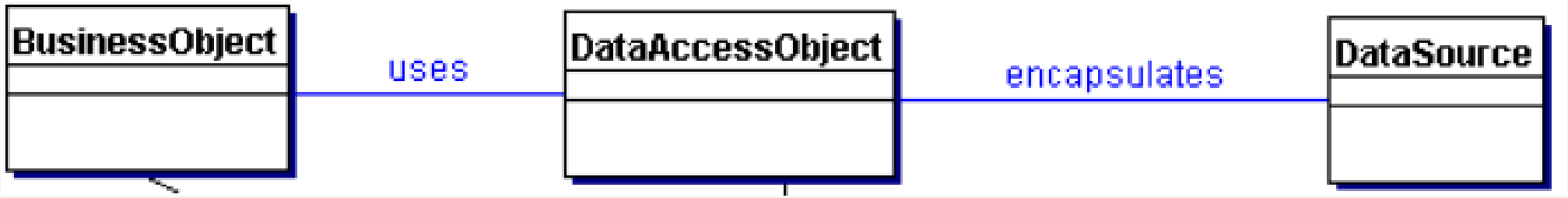
Siempre buscamos hacer nuestra aplicación lo más independiente posible de una base de datos concreta, de cómo se accede a los datos o incluso de si hay o no base de datos detrás. Nuestra aplicación debe conseguir los datos o ser capaz de guardarlos en algún sitio, pero no tiene por qué saber de dónde los está sacando o dónde se guardan.

Hay una forma de hacer esto que ha resultado bastante eficiente en el mundo JEE y de aplicaciones web, pero que es aplicable a cualquier tipo de aplicación que deba recoger datos de algún sitio y almacenarlos. Es lo que se conoce **como patrón DAO (Data Access Object).** La idea de este patrón es sencilla:

- **En primer lugar**, debemos hacernos las clases que representan nuestros datos. Por ejemplo, podemos hacer una clase Persona con los datos de la persona y los métodos set() y get() correspondientes.
- **En segundo lugar** hacemos un interfaz. Este interfaz tiene que tener los métodos necesarios para obtener y almacenar Personas. Este interfaz no debe tener nada que la relaciones con una base de datos ni cualquier otra cosa específica del medio de almacenamiento que vayamos a usar, es decir, ningún parámetro debería ser una Connection, ni un nombre de fichero, etc.

2.- Patrón DAO - Implementaciones

- Objetos de acceso a datos
  - Clases DAO con JDBC y SQL
- Frameworks de persistencia mapeo O/R (objeto/relación)
  - Hibernate, Spring, JPA (Java Persistence API)



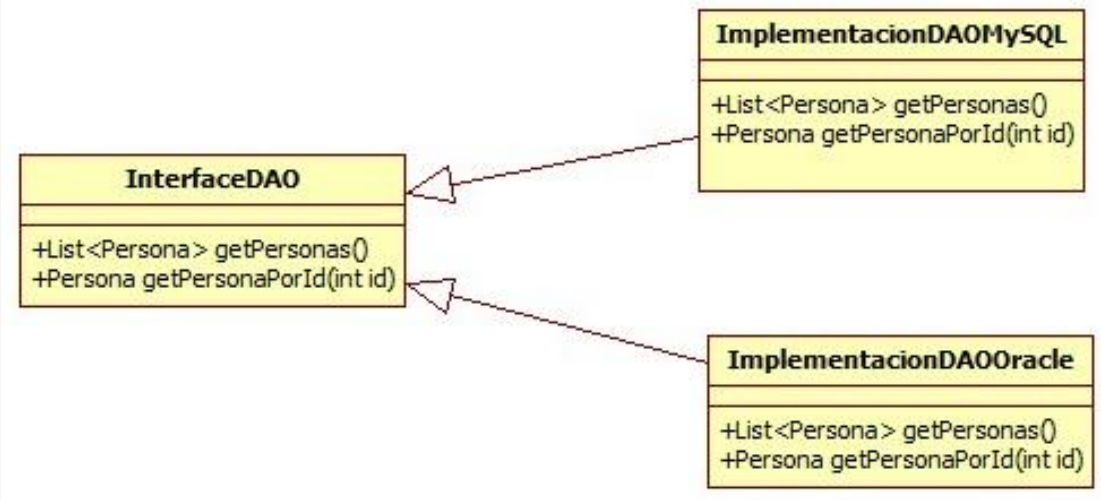
2.-Patrón DAO (II)

```
public interface InterfaceDAO {
   public List<Persona> getPersonas();
   public Persona getPersonaPorNombre (String nombre);
   ...
   public void salvaPersona (Persona persona);
   public void modificaPersona (Persona persona);
   ...
   public void borraPersonaPorNombre (String nombre);
   ...
}
```

Con esto deberíamos construir nuestra aplicación, usando la clase Persona y usando la InterfaceDAO para obtener y modificar Personas.

2.- Patrón DAO (III)

Aparte, hacemos la implementación de la InterfaceDAO, ya contra una base de datos concreta o usando una herramienta -iBATIS, Hibernate, etc- determinada. Al usar nuestra aplicación la InterfaceDAO, podremos pasarle cualquier implementación que queramos o incluso cambiar una por otra en un momento dado.



2.-Patrón DAO (IV)

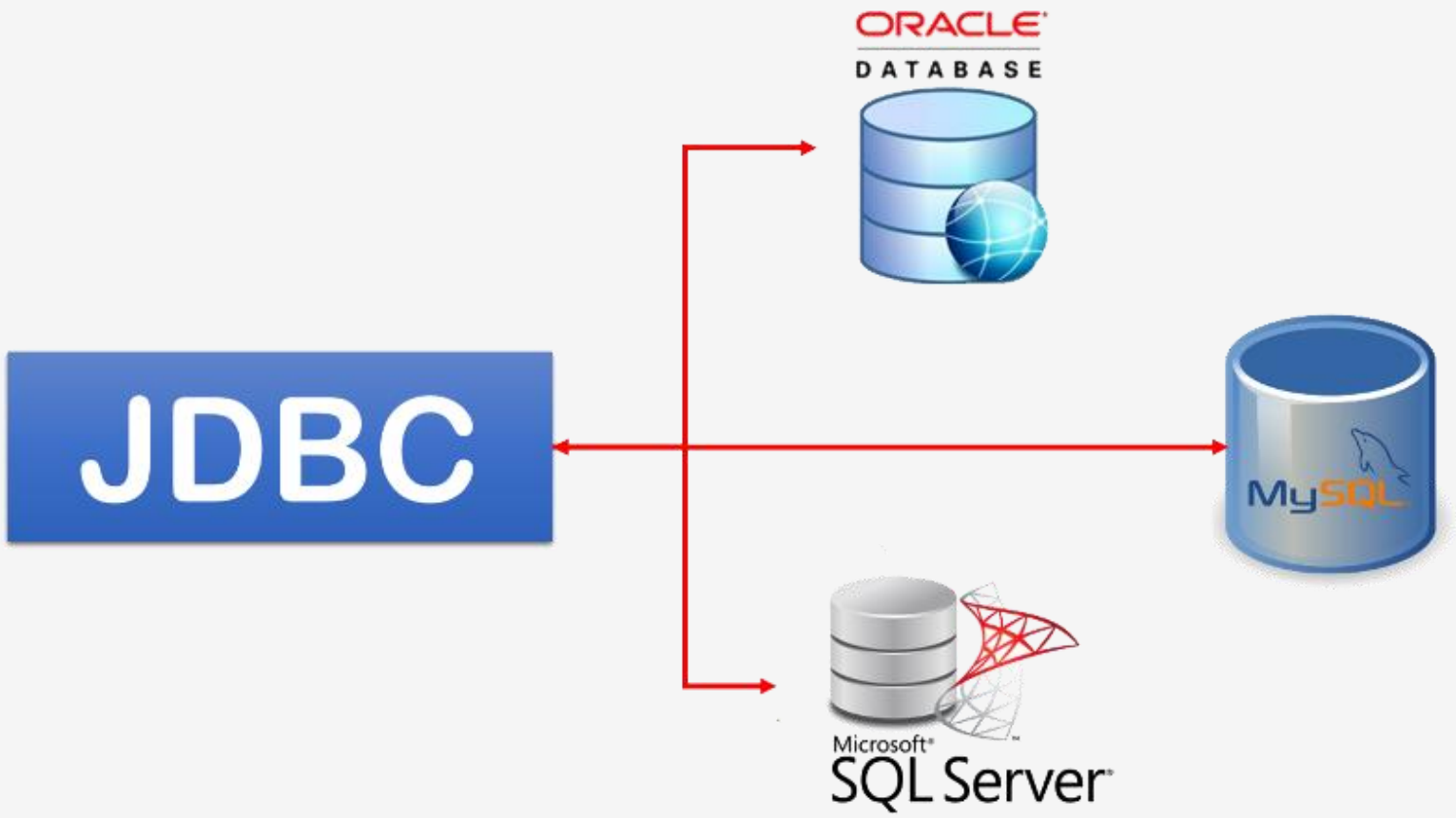
Normalmente el *patrón DAO* se completa con algún tipo de Factoría: Una clase a la que al pedirle la *InterfaceDAO* decide cual de las implementaciones instanciar y la devuelve.

```
public class FactoriaDAO
{
   public static InterfaceDAO getDAO() {
      if (baseDatos.equals("oracle")
         return new ImplementacionDAOOracle (parametrosConexionOracle);
      else if (baseDatos.equals("MySQL")
         return new ImplemetancionDAOMySQL (parametrosConexionMySQL);
      ...
   }
}
```

http://acodigo.blogspot.com/2016/03/data-access-object-dao-con-jdbc.html

3.-JDBC

- JDBC es el API de Java que nos permite acceder a datos en formato de tabla, en especial a datos almacenados en BD relacionales.



3.-JDBC

- JDBC es el API de Java que nos permite acceder a datos en formato de tabla, en especial a datos almacenados en BD relacionales.
- **JDBC incluye 4 componentes:**
  - **JDBC-API:** permite a las aplicaciones a través del lenguaje Java ejecutar sentencias SQL, recuperar datos y propagar cambios en un origen de datos (BD, FS)
    - Paquetes java.sql y javax.sql incluidos en Java SE y Java EE
  - **JDBC Driver Manager:** es el backbone de JDBC. Es la clase que nos pemite conectar una apliación Java a un driver JDBC
  - **JDBC Test Suite:** permite determinar que drivers puede testear la aplicación
  - **JDBC-ODBC Bridge:** Puente que nos permite acceder a driver ODBC via JDBC

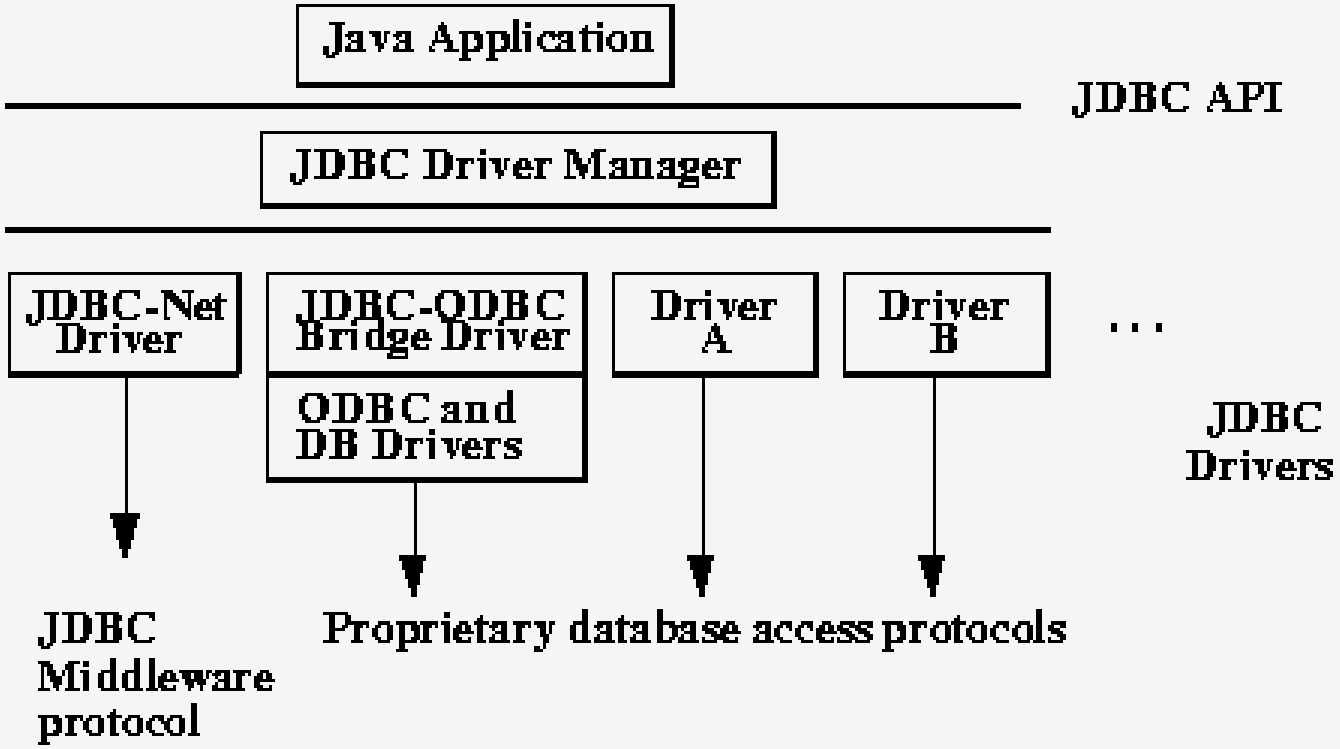
3.-JDBC - Establecer conexiones

JDBC para crear una conexión necesita usar una de las 2 clases siguientes:

- **DriverManager:** Clase implementada por completo que conecta una aplicación con un origen de datos especificado por un URL. Carga automáticamente los drivers JDBC 4.0 que encuentre en class path.

- **DataSource:** Interfaz que oculta los detalles de bajo nivel del origen de datos a la aplicación. Proporciona características avanzadas como “pool de conexiones” y transacciones distribuidas.
  - Se utiliza en aplicaciones Java EE.
  - Más complejo de utilizar

3.-Productos JDBC. Tipos de Drivers



4.Ejemplo JDBC Puro

Almacenamos los parámetros de conexión en un fichero XML.

mysql-properties.xml

```xml
<?xml version="1.0" encoding="UTF-8"?>
<!DOCTYPE properties SYSTEM "http://java.sun.com/dtd/properties.dtd">
<properties>

  <entry key="dbms">mysql</entry>
  <entry key="driver">com.mysql.jdbc.Driver</entry>
  <entry key="database_name">mercado</entry>
  <entry key="user_name">root</entry>
  <entry key="password"></entry>
  <entry key="server_name">localhost</entry>
  <entry key="port_number">3306</entry>


</properties>
```

4.Ejemplo JDBC Puro

Establecemos métodos para abrir y cerrar la conexión.

```java
81    public Connection getConnection() throws SQLException {
82
83        Connection conn = null;
84        Properties connectionProps = new Properties();
85        connectionProps.put("user", this.userName);
86        connectionProps.put("password", this.password);
87
88        if (this.dbms.equals("mysql")) {
89            /* Solicito a DriverManager una conexión con la base de datos */
90            /*
91             * Para identificar el controldador a usar se le proporciona una
92             * URL, si no lo encuentra lanza SQLException
93             */
94            /* formato de URL: jdbc:[host][:port]/[database] */
95            /*
96             * La URL varía según el gestor de BD,
97             * jdbc:mysql://127.0.0.1:3306/libros,
98             * jdbc:oracle:thin:@192.168.239.142:1521:libros
99             */
100           conn = DriverManager.getConnection("jdbc:" + this.dbms + "://"
101                   + this.serverName + ":" + this.portNumber + "/" + this.dbName,
102                   connectionProps);
103       } else if (this.dbms.equals("derby")) {
104           conn = DriverManager.getConnection("jdbc:" + this.dbms + ":"
105                   + this.dbName + ";create=true", connectionProps);
106       }
107       System.out.println("Connectado a BD");
108       return conn;
109   }
```

```java
111   /**
112    * Cierre de conexión a BD
113    * @param connArg
114    */
115   public static void closeConnection(Connection connArg) {
116       System.out.println("Releasing all open resources ...");
117       try {
118           if (connArg != null) {
119               connArg.close();
120               connArg = null;
121           }
122       } catch (SQLException sqle) {
123           printSQLException(sqle);
124       }
125   }
```

4.Ejemplo JDBC Puro

Establecemos las queries que vamos a lanzar, estas con la sintaxis de sistema gestor de bases de datos que estamos empleando: Oracle, MySQL…

```
public class CafeJDBC implements CafeDAO{


    // Consultas a realizar en BD
    private static final String SELECT_CAFES_QUERY = "select CAF_NOMBRE, PROV_ID, PRECIO, VENTAS, TOTAL from CAFES";
    // En una consulta parametrizada ponemos interrogaciones en los valores que
    // aﹾn desconocemos
    private static final String UPDATE_VENTAS_CAFE = "update CAFES set VENTAS = ? where CAF_NOMBRE = ?";
```

4.Ejemplo JDBC Puro

```java
public void verTabla() {
    /* Conexi□n a la Base de Datos */
    Connection con = null;
    /* Sentencia sql */
    Statement stmt = null;
    /* Conjunto de Resultados a obtener de la sentencia sql */
    ResultSet rs = null;
    try {
        con = new Utilidades().getConnection();
        // Creaci□n de la sentencia
        stmt = con.createStatement();
        // Ejecuci□n de la consulta y obtenci□n de resultados en un
        // ResultSet
        rs = stmt.executeQuery(SELECT_CAFES_QUERY);

        // Recuperaci□n de los datos del ResultSet
        while (rs.next()) {
            String coffeeName = rs.getString("CAF_NOMBRE");
            int supplierID = rs.getInt("PROV_ID");
            float PRECIO = rs.getFloat("PRECIO");
            int VENTAS = rs.getInt("VENTAS");
            int total = rs.getInt("TOTAL");
            System.out.println(coffeeName + ", " + supplierID + ", "
                    + PRECIO + ", " + VENTAS + ", " + total);
        }

    } catch (IOException e) {
```

Iniciamos la conexión de BD

Creamos un statement, para lanzar la consulta

Seleccionamos la query que queremos lanzar.

Obtenemos los datos de un ResultSet, donde el parámetro se corresponde con el nombre de la columna de la BD.

4.Ejemplo JDBC Puro

```java
} catch (IOException e) {
    // Error al leer propiedades
    // En una aplicaci□n real, escribo en el log y delego
    System.err.println(e.getMessage());
} catch (SQLException sqle) {
    // En una aplicaci□n real, escribo en el log y delego
    System.err.println(sqle.getMessage());
} finally {
    try {
        // Liberamos todos los recursos pase lo que pase
        if (rs != null) {
            rs.close();
        }
        if (stmt != null) {
            stmt.close();
        }
        if (con != null) {
            Utilidades.closeConnection(con);
        }
    } catch (SQLException sqle) {
        // En una aplicaci□n real, escribo en el log, no delego porque
        // es error al liberar recursos
    }
}
```

Gestionamos las posibles excepciones que hayan hecho finalizar la operación

Cerramos la conexión para casos satisfactorios o erróneos

5.-Spring

El anterior ejemplo es una implementación de JDBC puro, en la que es el programador que el que gestiona la apertura y el cierre de la conexión.

Esta implementación generar problemas como que se satura el pool de conexiones, cerrar la conexión inadecuadamente.

Las implementaciones actuales de JDBC corrigen esto añadiendo un **Transaction Manager**. Para ver como funciona tendremos que ver ciertos aspectos de funcionamiento interno de Spring.

6.-JDBC Template

A continuación vamos ver una implementación actual de JDBC en Spring, relegando el mapeo de datos a la apertura y cierre de conexiones al Framework.

Como siempre empezamos añadiendo la librerías necesarias al pom.xml

```
<dependency>
    <groupId>mysql</groupId>
    <artifactId>mysql-connector-java</artifactId>
    <version>8.0.21</version>
</dependency>
    <dependency>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-starter-jdbc</artifactId>
</dependency>
```

6.- JDBC Template

A continuación necesitamos configurar el **DataSource** es decir, establecer los datos de la conexión.

Para ello añadimos la configuración al **application.properties**.

```
server.port=9090
spring.thymeleaf.cache=false

# MySQL
spring.datasource.driver-class-name=com.mysql.jdbc.Driver
spring.datasource.url=jdbc:mysql://localhost/springbootjdbc?allowPublicKeyRetrieval=tr
ue&createDatabaseIfNotExist=true&useUnicode=true&useJDBCCompliantTimezoneShift=true&us
eLegacyDatetimeCode=false&serverTimezone=UTC&autoReconnect=true&useSSL=false
spring.datasource.username=root
spring.datasource.password=root
spring.datasource.initialization-mode=always
```

6.- JDBC Template

Una vez creado los datos para la conexión creamos el objeto java con del que mapearemos datos de la BD, esto se denomina POJO.

POJO (**Plain Old Java Object**) es una instancia de una clase que no extiende ni implementa nada en especial. Es decir una clase sencilla con sus atributos gettters y setters.

```
public class Book {
    private Long id;
    private String name;
    private BigDecimal price;

    public Book(Long id, String name, BigDecimal price) {
        this.id = id;
        this.name = name;
        this.price = price;
    }
    //getters y setters
```

6.- JDBC Template

Creamos el Dao.

```
public interface BookDao {

    int count();
    int save(Book book);
    int update(Book book);
    int deleteById(Long id);
    List<Book> findAll();
    List<Book> findByNameAndPrice(String name, BigDecimal price);
    Optional<Book> findById(Long id);
    String getNameById(Long id);
}
```

6.- JDBC Template

Implementamos el Dao. Importante añadir la anotación @Repository para poder detectar que esta clase hace uso de conexiones a BD.

El método sabe llama a **jdbcTemplate.update** para que modifique la BD.

```
@Repository
public class BookJdbc implements BookDao {

    @Autowired
    private JdbcTemplate jdbcTemplate;

    @Override
    public int save(Book book) {
    return jdbcTemplate.update("insert into books (name, price) values(?,?)",
    book.getName(), book.getPrice());
    }
```

6.- JDBC Template

El método sabe llama **a jdbcTemplate.query** permite lanzar una consulta que nos devuelva datos. Los datos serán mapeador directamente al objeto Book que es nuestro POJO

```
@Override
public List<Book> findAll() {
return jdbcTemplate.query("select * from books",
(rs, rowNum) -> new Book(rs.getLong("id"), rs.getString("name"),
rs.getBigDecimal("price")));
}
```

6.- JDBC Template

Llamamos al Dao.

Para instanciar el Dao hacemos uso de la anotación **@Autowired** para que el sepa resolver la implementación de BookDao,

```
@Controller
public class BookController {

    @Autowired
    private BookDao bookdao;

    @RequestMapping(value="/", method=RequestMethod.GET)
        public String index(Model modelo) {
        List<Book> libros = bookdao.findAll();
        modelo.addAttribute("libros",libros);
        return "libros";
    }
```

6.- JDBC puro vs JDBC implementado en Spring

```java
public void verTabla() {
    /* Conexi□n a la Base de Datos */
    Connection con = null;
    /* Sentencia sql */
    Statement stmt = null;
    /* Conjunto de Resultados a obtener de la sentencia sql */
    ResultSet rs = null;
    try {
        con = new Utilidades().getConnection();
        // Creaci□n de la sentencia
        stmt = con.createStatement();
        // Ejecuci□n de la consulta y obtenci□n de resultados en un
        // ResultSet
        rs = stmt.executeQuery(SELECT_CAFES_QUERY);

        // Recuperaci□n de los datos del ResultSet
        while (rs.next()) {
            String coffeeName = rs.getString("CAF_NOMBRE");
            int supplierID = rs.getInt("PROV_ID");
            float PRECIO = rs.getFloat("PRECIO");
            int VENTAS = rs.getInt("VENTAS");
            int total = rs.getInt("TOTAL");
            System.out.println(coffeeName + ", " + supplierID + ", "
                    + PRECIO + ", " + VENTAS + ", " + total);
        }

    } catch (IOException e) {
```

```java
@Override
public List<Book> findAll() {
return jdbcTemplate.query("select * from
books",
(rs, rowNum) -> new Book(rs.getLong("id"),
rs.getString("name"),
rs.getBigDecimal("price")));
}
```